

Relazione progetto Programmazione a Oggetti

Canazza Angela, mat. 2111030
Sadè Linor, mat. 2111942

Titolo del progetto: *Cinema Manager*

1 Introduzione

Il progetto presenta il concetto di una libreria multimediale progettata come gestionale per una catena di cinema. I media sono rappresentati da ciò che si può trovare sul grande schermo dei cinema: i film, i trailer e le inserzioni pubblicitarie. Inoltre sono inclusi tra i media anche i podcast e le loro puntate che la catena di cinema distribuisce sui propri canali. Per ogni cinema, l'applicazione consente di visualizzare i media, modificarne le caratteristiche, cercarli (attraverso titolo o autore) e cancellarli.

L'idea che sta alla base del progetto si ispira alla modalità con cui i principali cinema programmano la proiezione e distribuzione dei propri contenuti. Per esempio, recandosi sul sito di *The Space Cinema* si troveranno difficilmente gli orari di programmazione di film oltre la settimana corrente perchè spesso la loro proiezione futura dipende dalle vendite passate (salvo casi particolari).

Il gestionale rende semplice la supervisione e la scelta di estendere la programmazione alle settimane o mesi successivi attraverso anche la semplicità di utilizzo e la grafica moderna. Le modalità con cui gli incassi vengono calcolati o le date vengono estese per ogni tipologia di contenuto sfrutta il polimorfismo della gerarchia dei media.

2 Descrizione del modello

L'UML, mostrato in [Figura 1](#), illustra come sono stati modellati i media della gerarchia e le classi per la gestione dei file strutturati. Per questo progetto sono state previste 8 classi per i media: 2 astratte e 6 concrete. Tutti i media derivano dalla classe base astratta **Media**, che rappresenta un generico contenuto multimediale del cinema. Le classi **Film**, **Podcast** e **Puntata** derivano direttamente da **Media**, mentre i **Trailer** e le **Inserzioni** pubblicitarie discendono dalla classe astratta **Pubblicità**, che riporta il numero di proiezioni giornaliere stabilite per quel determinato media.

Ogni media possiede, oltre agli attributi naturali come Titolo, Autore e Descrizione, anche il numero di visualizzazioni. Ai fini accademici del progetto, poiché l'informazione delle visualizzazioni viene utilizzata nell'implementazione di alcuni metodi virtuali, è stato introdotto il metodo `incrementaVisualizzazioni()`, che simula l'aumento delle visualizzazioni in base ai giorni in cui il media è attivo. Naturalmente tale dato in una situazione realistica verrebbe prelevato nel momento del bisogno da un database costantemente aggiornato. Inoltre, ciascun contenuto del cinema è caratterizzato da una data di inizio distribuzione (detta di inizio rilascio) e da una data di fine distribuzione (detta di fine rilascio).

Il calcolo dell'incasso (`calcolaIncasso()`) e la modalità di **estensione della data di fine distribuzione** (`estendiDataFineRilascio()`) differiscono a seconda del tipo di media. In ogni caso, **l'estensione è valida solo se il media si trova ancora in stato di distribuzione.**

Per i **Film**, l'incasso è dato dalla moltiplicazione tra il numero di visualizzazioni e il costo del biglietto. I film hanno un vettore di **Trailer** e, per mantenere un forte legame tra le due classi, nella classe **Trailer** è stato introdotto un puntatore al film di riferimento. Poiché un film è proprietario dei suoi trailer, ha anche il compito di eliminarli quando viene distrutto. L'incasso associato ai trailer non è reale, ma rappresenta una stima delle possibili vendite generate dalla loro visione, calcolata tramite il metodo `calcolaTassoDiStima()`, che considera caratteristiche del film come valutazione, genere e classificazione. La programmazione dei film è gestita settimanalmente, quindi l'estensione della data di fine rilascio avanza di 7 giorni ad ogni invocazione del metodo. Naturalmente se i film rimangono in sala, l'avanzamento della data di fine rilascio viene propagato anche ai trailer associati.

Un'**Inserzione**, invece, è una pubblicità commissionata da un'azienda per essere riprodotta al cinema nelle fasce orarie indicate. Da contratto, include sempre la fascia oraria mattutina, mentre le altre fasce

comportano un incremento sul costo base. L'estensione della data di fine rilascio per le inserzioni avanza di un mese. Il calcolo dell'incasso invece dipende dal numero di proiezioni giornaliere stabilite, il costo fisso per proiezione, il numero e la tipologia di fasce orarie e la durata della proiezione nei canali della catena di cinema.

I **Podcast** e le **Puntate** seguono lo stesso principio dei film e dei trailer: un Podcast è "proprietario" delle puntate. Il calcolo degli incassi del podcast proviene dalla somma dei calcoli delle singole puntate che lo compongono e l'incasso delle puntate è calcolato sulla base delle visualizzazioni, un costo fisso per pubblicità e il numero di pubblicità. La data di fine rilascio (risp. inizio rilascio) del podcast corrisponde alla data di fine rilascio maggiore (risp. minore) tra le puntate del podcast. L'estensione della data di fine distribuzione del podcast estende di un giorno quella delle sue puntate. Le puntate possono essere singolarmente estese di un giorno e ciò potrebbe comportare l'estensione anche del podcast.

La classe **Cinema** rappresenta un contenitore che memorizza tutti i contenuti multimediali associati al cinema. I contenuti vengono gestiti tramite una lista, nella quale non possono essere inseriti due oggetti **Media** con lo stesso titolo e autore, che costituiscono quindi una chiave di univocità

Nello sviluppo del modello logico, un'attenzione particolare è stata posta al mantenimento di invarianti, come:

- un trailer è al più collegato ad un solo film
- Un film contiene solo trailer associati a sé stesso e mai duplicati (stessa invariante tra Podcast-Puntata)
- la data di fine rilascio di un trailer non può mai superare quella del film
- la data di fine rilascio di un podcast non può mai essere inferiore alla data di fine rilascio delle sue puntate (e in particolare è sempre uguale a quella della puntata con fine rilascio maggiore)
- la fascia oraria "Mattina" è sempre presente in un'inserzione
- in ogni stato dell'applicazione, le visualizzazioni del podcast sono sempre la somma delle visualizzazioni delle puntate ad esso associate
- lingua e sottotitoli del podcast sono l'unione di quelli delle puntate
- le date di inizio e fine distribuzione del podcast rispettano sempre la data minima e massima delle puntate ad esso associate

Si noti che il mantenimento di una relazione stretta tra film e trailer e tra podcast e puntate introduce complessità nella gestione. Tale complessità risulta più evidente nella sezione [4](#).

3 Polimorfismo

Sono state implementate diverse funzioni di polimorfismo all'interno del progetto.

Tra le principali si vede l'uso del design pattern visitor:

- **CardVisitor:** utilizzato per generare il widget **MediaFrame** o **HorizontalCard** a seconda della visualizzazione scelta (vista compatta o espansa). Il Visitor in questo caso non distingue il tipo concreto di **Media**, infatti il widget generato è lo stesso per tutti i tipi. La sua funzione principale è quella di applicare il filtro selezionato (per esempio in `visit(Film*) filtro == "Film"`) in modo da evitare l'uso di `dynamic_cast` espliciti nel ciclo di costruzione della libreria dei widget media.
- **DetailPageVisitor:** utilizzato per generare un'intera pagina di tipo statico **MediaView** che viene mostrata per la visualizzazione di un media. In particolare **MediaView** è una classe base astratta da cui ereditano le specifiche classi **FilmView**, **TrailerView**, **InserzioneView**, **PodcastView**, **PuntataView**.



- **JsonVisitor / XmlVisitor:** utilizzati per trasformare un oggetto **Media** in una rappresentazione esterna (rispettivamente **QJsonObject** per JSON o **QDomElement** per XML) per la serializzazione dei dati.

Un altro impiego del polimorfismo si trova nell'implementazione del pattern Observer. **SearchPanel** registra un vettore di **LibraryObserver** che vengono notificati (tramite l'invocazione di **update()**) quando l'utente interagisce con i criteri di ricerca e visualizzazione come filtri, ricerca, ordinamento, tipologia di vista. La classe **UpdateMediaLibrary** eredita direttamente da **LibraryObserver** e fornisce un'implementazione concreta del metodo **update**.

3.1 DetailPageVisitor

Il visitor non genera singoli widget, ma istanzia direttamente l'intera pagina (**MediaView**) corrispondente al media. Sebbene la **UI** possa sembrare simile tra le varie pagine, è stata progettata con alcune differenze che hanno reso complessa la possibile creazione di widget specifici per ciascun tipo di media utilizzando il design pattern Visitor in modo classico, da qui la scelta di creare l'intera pagina. Quindi, per evitare duplicazione di codice è stata introdotta la classe **MediaView** che definisce le parti della pagina comuni tra i diversi tipi di media e obbliga le sottoclassi a implementare i metodi necessari per popolare (o creare) i widget specifici del singolo tipo.

Inoltre lo stesso principio è stato utilizzato per l'update della pagina. Lo slot pubblico virtuale **MediaView::update()** aggiorna la visualizzazione dei widget comuni per ogni sottotipo e ognuno di essi reimplementa il metodo invocando l'omonimo metodo genitore e aggiungendo l'aggiornamento dei widget specifici. Diventa quindi sufficiente chiamare **update** su un puntatore polimorfo **MediaView** per aggiornare correttamente un qualsiasi suo sottotipo.

Questo tipo di struttura ha permesso di sfruttare il polimorfismo anche nei segnali: in **SearchPanel** è sufficiente connettere un singolo segnale comune a tutti i tipi **MediaView** per poter eseguire una specifica azione. Questo riduce l'accoppiamento e centralizza la logica di gestione eventi (è il caso di **requestMediaView(MediaView& widget)**, **editMediaClicked(Media*)**, **extendMediaClicked()**, **deleteMediaClicked(Media*)**, **returnbutton()**).

Tuttavia per evitare **static_cast** (che avrebbe reso inutile lo scopo stesso del visitor), ogni sottoclasse di **MediaView** introduce, oltre al tipo generico **Media*** ereditato, anche un puntatore al tipo concreto corretto (ad esempio **Film*** in **FilmView**). Grazie al modo in cui è stato progettato il modello, è sempre garantito che i due puntatori puntino allo stesso oggetto.

4 Persistenza dei dati

La persistenza dei dati è gestita tramite il formato strutturato **Json**. All'apertura dell'applicazione, essa carica automaticamente il file **media.json** che si trova all'interno della cartella **Json_XML** nella root del progetto, altrimenti il file viene creato senza contenuti.

Se il file contiene già dei **Cinema** con i relativi media, gli oggetti **Cinema** (costituiti da nome e path dell'immagine) vengono costruiti e resi disponibili all'apertura dell'applicazione. Una volta aperto un cinema, i suoi media vengono letti da **s_jsonManager** e convertiti in oggetti **Media*** tramite la funzione **loadMedia(QList<Media*>&, const QString&)**. Infine gli oggetti vengono raccolti in una **QList** temporanea **s_MediaListOfCinema**. In alternativa, l'utente può creare un nuovo cinema (che verrà salvato in **media.json**) e successivamente aggiungervi i media desiderati.

I media vengono quindi allocati in memoria solo quando si entra all'interno di un cinema e all'uscita vengono deallocati. Queste considerazioni sono state fatte per ottimizzare l'uso della memoria. Il processo inizia ad ogni cambio cinema.

Dal punto di vista implementativo, il contenitore **s_MediaListOfCinema** svolge il ruolo di lista qt-friendly di supporto condivisa tra **SearchPanel**, le librerie e i pannelli di inserimento e modifica. Al primo caricamento di un cinema, attraverso **updateInfoCinema**, gli oggetti vengono letti dal file JSON con **loadMedia()** e inseriti sia in **s_MediaListOfCinema** sia nel cinema selezionato tramite **addMedia()**. In questa fase, dunque, **s_MediaListOfCinema** funge da sorgente per popolare il cinema. Una volta caricato, però, il suo compito non è più quello di popolare il cinema, ma è quello di mantenere una copia parallela dei puntatori agli stessi oggetti: ciò consente alle librerie di leggere direttamente da essa, alle funzioni di filtro e ricerca di operare senza accedere al cinema e ai pannelli di inserimento di aggiornare

coerentemente entrambe le strutture, poiché la lista è passata per riferimento. Infine, in fase di reset (ossia all'uscita di un cinema oppure all'eliminazione di un cinema) la lista viene svuotata e gli oggetti distrutti (`resetSearchPanel`) oppure semplicemente liberata dai puntatori se la distruzione è già stata effettuata altrove (`resetSearchPanelAfterDeleteCinema`), garantendo così l'assenza di dangling pointers.

Oltre al JSON, l'applicazione supporta anche il formato XML per l'importazione ed esportazione di media o interi cinema (sessioni). Ad esempio, dalla Menu bar:

- è possibile esportare i media di un cinema e importarli in un altro (a meno di omonimia);
- è possibile condividere cinema tra istanze della stessa applicazione (o per creare dei semplici backup)

Poiché lo stato dell'applicazione è sempre gestito dal file JSON, l'importazione in Xml invoca i metodi dedicati alla persistenza in json per completare le operazioni che vengono delegate al JSON manager (`CinemaRepositoryJson`) mentre l'XML viene utilizzato come formato di scambio o backup.

Il manager XML, rappresentato dalla classe `MediaManagerXml`, gestisce le operazioni di importazione e esportazione dei cinema in formato XML. Ogni volta che un cinema viene selezionato il segnale viene mandato a `SearchPanel` nello slot `updateInfoCinema` e `s_xmlManager` aggiorna lo stato corrente della sessione tramite il metodo `setCurrentCinema(Cinema*)`.

Si vuole far notare la complessità citata nella sezione 2. Il modello logico adottato garantisce che istanze di Trailer associate ad un determinato Film non possano comparire nei vettori di Trailer di un'altra istanza di Film, pertanto nella rappresentazione in formato JSON (o XML) è stato sufficiente memorizzare i Trailer come entità distinte, associandoli al rispettivo Film senza che il legame fosse bidirezionale a livello di file.

Tuttavia l'associazione bivalente tra Film-Trailer e Podcast-Puntata ha reso complessa la gestione delle dipendenze sia in fase di creazione dei media sia in fase di eliminazione degli stessi:

- per poter creare un Trailer è necessario che il Film corrispondente sia già stato creato e scritto nel file JSON; in maniera analoga, per una Puntata è necessario che il Podcast esista già;
- l'eliminazione di un Film o di un Podcast richiede di gestire esplicitamente il disaccoppiamento dei Trailer o delle Puntate a essi associati. Questo perché l'operazione di eliminazione consisterebbe nell'invocazione di `delete` per ciascun puntatore `Media*` della lista di media. Tuttavia, poiché Film è proprietario dei propri trailer (e un podcast delle proprie puntate) la distruzione di film comporterebbe automaticamente la distruzione dei relativi trailer. Di conseguenza, i puntatori presenti nella lista che facevano riferimento a quei Trailer (o Puntate) diventerebbero dangling pointer.

L'associazione tra entità **Film-Trailer** e **Podcast-Puntata** è realizzata nel file JSON (e XML) tramite la coppia di chiavi (Titolo, Autore) di Film e Podcast che garantisce un riferimento univoco al media nello stesso cinema. Nella costruzione degli oggetti Trailer e Puntata, i riferimenti (indirizzi di memoria) a Film e Podcast vengono fatte con funzioni come `findMediaReference` (in `JsonConverter`) o `findFilmInList` `findPodcastInList` (in `XmlReader`).

Oltre al file principale `media.json`, nella cartella `Json_XML` sono presenti anche due file in formato XML:

- **CinemaNotorius.xml**: rappresenta un backup completo di un cinema già memorizzato nel file JSON ed è importabile tramite la funzione `Importa sessione` (a meno di omonimia)
- **altri_media.xml**: un file contenente alcuni media aggiuntivi non presenti nel file JSON, importabile tramite la funzione `Importa media`

5 Funzionalità aggiuntive implementate

- Gestione di cinema:
 - Gestione di una catena di cinema piuttosto che di un singolo cinema
 - Possibilità di modificare le informazioni del cinema (nome e copertina) dopo la creazione
 - Possibilità di eliminare un cinema (tramite Menu)

- Immagini di default per i cinema quando non vengono caricate
- Gestione dei media:
 - Simulazione di valutazioni e visualizzazioni realistiche (se troppo alte, la scusa è che la catena di cinema è molto popolare). In particolare:
 - * L'assegnazione di visualizzazioni e valutazioni dipende dalla randomicità durante la creazione dell'oggetto e dalla data di creazione (ossia quella corrente)
 - * Gli oggetti vengono ricreati ogni volta che si accede al singolo cinema a cui sono assegnati, garantendo un "aggiornamento" costante delle visualizzazioni.
 - * In alcuni casi le visualizzazioni potrebbero risultare minori, ma ai fini del progetto questo dettaglio è stato trascurato.
 - Possibilità di aggiungere un media senza scegliere a priori il tipo di media: la pagina si adatta dinamicamente al valore di "Tipologia"
 - La pagina di visualizzazione di un media permette di navigare tra i media correlati e tornare alla pagina precedentemente visualizzata, garantendo una gestione efficiente della memoria
 - Immagini di default per i media quando non vengono caricate, variabili per tipologia di media
- Persistenza e import/export:
 - Persistenza automatizzata dei dati ad ogni modifica salvata (formato Json)
 - Backup dei dati e condivisione di liste di media o sessioni di cinema (formato Xml)
 - Scelta della modalità di importazione ed esportazione (formato Xml)
 - Controllo minimo degli errori nell'importazione Xml (es. film non trovato per un trailer)
- Interfaccia utente e interazione:
 - Dialog personalizzate che comunicano anticipatamente o successivamente l'effetto di un'azione
 - Sistema di filtri di ricerca combinati:
 - * Ricerca per titolo o autore
 - * Attività: attivo, fuori produzione o tutto
 - * Popolarità basata sulle visualizzazioni
 - * Data di uscita: recenti o meno recenti
 - Cambio vista per anteprima più estesa delle informazioni del media
 - Uso di widget custom in molti casi
 - Override di `resizeEvent` per mantenere una visione minima in caso di ridimensionamento
 - Particolare attenzione a UI e UX:
 - * Vie alternative per raggiungere la stessa funzione (es. uscita dal cinema tramite menu o pulsante)
 - * Attenzione ai colori (es. rosso per azioni pericolose)
 - * Menu personalizzato con azioni disattivate se non applicabili (es. importazione media prima di entrare in un cinema)
 - * Pulsanti estendi ed elimina con toolbutton rispettivi per disattivare i pulsanti che suggeriscono all'utente che è un'operazione per cui prestare attenzione.
Nota: la modifica diretta della data di fine è comunque possibile nella pagina di modifica dei media ma rimangono le invarianti descritte nella sezione 2.
 - * Widget custom per inserimento immagini con possibilità di rimozione
 - * Visualizzazione delle informazioni in ordine logico per ogni tipologia di media
 - * Widget custom di label espandibile per informazioni lunghe o opzionali
 - * Controllo disponibilità del nome nella creazione/modifica di cinema
 - * Azioni disattivate in base al modello logico (es. date non previste)
 - * Visualizzazione di Nessuno o Nessuna se alcuni dati non sono valorizzati
- Modalità e visualizzazione:
 - Modalità full screen
 - Utilizzo di immagini di default quando il percorso specificato non produce risultati (nullptr) o quando l'utente decide di non inserirle.
 - Immagini di default differenziate per tipologia di media, con una specifica anche per i cinema.

6 Rendicontazione ore

Attività	Ore Previste	Ore Effettive
Studio e progettazione	14	20
Test e debug	5	4
Sviluppo del codice del modello	10	16
Studio del framework Qt	8	10
Sviluppo del codice della GUI	30	25
Sviluppo della grafica su Figma	5	9
Abbellimenti tramite QSS	6	3
Stesura della relazione	4	4
Totale	82	91

La tabella mostra che le ore effettive per il progetto si avvicinano a quelle previste ma restano comunque al di sopra della richiesta di 40 ore (per persona). Il motivo di questo scostamento ha 4 principali cause. In primo luogo, l'inesperienza ci ha portato a sottovalutare alcune scelte architetturali, e allo stesso tempo, a sopravvalutarne altre. Infatti si nota che lo studio e la progettazione hanno richiesto molto tempo, poichè eravamo consapevoli che eventuali cambi di rotta avrebbero comportato ritardi maggiori; in alcuni casi questo approccio si è rivelato vantaggioso, mentre in altri ha portato ad un'estensione dei tempi senza benefici (per esempio l'introduzione di una documentazione Doxygen che non è stata più implementata nella parte della GUI).

Inoltre il progetto è stato accantonato in due occasioni, per essere ripreso successivamente, rompendo il ritmo e causando un *overhead* ad ogni "ripresa" del progetto. Un altro motivo è che abbiamo avuto particolari problemi nella progettazione delle classi per la persistenza dei dati aggravato dall'introduzione tardiva della classe `Cinema` nel modello. Infine la complessità discussa nella sezione 4 ha rallentato ulteriormente i tempi di completamento del progetto.

7 Suddivisione delle attività tra i membri del gruppo

Attività	Responsabile
Sviluppo del codice del modello logico per: <i>Media, Pubblicità, Inserzione, Film, Trailer</i>	Linor Sadè
Sviluppo del codice del modello logico per: <i>Cinema, Podcast, Puntata</i>	Angela Canazza
<code>EnumClasses</code>	Linor Sadè
Test	Entrambe
Widget personalizzati	Entrambe (a seconda di chi ne aveva bisogno)
<code>SearchPanel</code> e <code>MainWindow</code>	Entrambe
<code>CinemaSelectionPage</code> e <code>InsertCinemaPage</code>	Linor Sadè (principalmente)
<code>MediaEditor</code> (classi per l'aggiunta e modifica di un media)	Angela Canazza
<code>MediaViewPages</code> (classi per la visualizzazione di un media)	Linor Sadè
<code>Library</code> (classi per i filtri nella visualizzazione dei media)	Angela Canazza
Gestione del JSON	Angela Canazza
Gestione dell'XML	Linor Sadè
Inserimento dei media di prova nei file JSON e XML	Entrambe
Progettazione grafica su Figma	Linor Sadè
Abbellimenti tramite QSS	Entrambe (a seconda della classe sviluppata)

Ogni attività è stata discussa in anticipo per individuare la soluzione migliore. Eventuali problemi o proposte di miglioramento sono sempre stati condivisi da entrambe e risolti dal membro responsabile dell'attività.